Distilled Command Parser on a Local Cloud — Report

Devarayi Thanuj, 230101003 · Cloud Computing (CS3081) · https://github.com/thanuj85/cloud_assignment

Every number below was produced by running the code in this repository on the machine described in Section 1. Where a measurement has not been taken yet, the row says *pending* and gives the exact command that produces it, rather than a made-up value.

1. Setup

Item	Value
Machine	Apple silicon laptop, 8 GB RAM, CPU only, macOS
Python	3.11.14
torch / transformers	2.1.2 / 4.36.2
Teacher	`phi3.5:latest` via Ollama, 2.2 GB, temperature 0, seed 42, `num_ctx 2048`
Student	`EleutherAI/pythia-70m`, 70M parameters, Apache 2.0
Cluster	k3d, 1 server + 1 agent, k3s with Traefik and metrics-server
Storage	SeaweedFS in namespace `data`, S3 endpoint on port 8333

2. Task 1 — Object storage and parallel teacher labelling

`cluster/k3d.yaml` defines the cluster; `infra/` creates the namespaces `data`, `pipeline` and `app`, deploys SeaweedFS, and `infra/make_buckets.py` creates the `datasets` and `models` buckets and uploads `commands_pool.jsonl` to `datasets`.

Prompt. `prompts/teacher_prompt.txt` asks for one JSON object with exactly the four keys, requires slot text to be copied verbatim (no `07:00` style conversion), requires `null` for an unmentioned slot, and forbids prose or code fences. The command is substituted for the `{command}` placeholder.

Sharding. `worker/label.py` reads `SHARD_INDEX` and `SHARD_COUNT` and keeps the commands whose line number *L* (from 0) satisfies `L mod SHARD_COUNT == SHARD_INDEX`. Because every line has exactly one remainder, the shards are disjoint and together cover the pool; `scripts/check_shards.py` asserts that property (no gaps, no repeats, nothing outside the pool). Each worker writes its own object, `datasets/teacher/shard-N.jsonl`, and stores the teacher's reply byte-for-byte in `teacher_raw`. The worker is resumable: on restart it reads back its own object and skips ids already present, which matters for a run measured in hours.

Measured labelling rate. Two shards ran at once, 6 commands each:

```
shard 0/2: 1500 commands, 0 done, 6 to label      6/6 (0.20/s)
shard 1/2: 1500 commands, 0 done, 6 to label      6/6 (0.19/s)
```

0.2 commands/s per shard → 3,000 commands ≈ 15,000 s ≈ 4.2 h on one worker, ≈ 1.05 h with four parallel shards. That is the dominant cost of the project on a CPU-only laptop.

Two measurements that changed the prompt/worker design. First, phi3.5 sometimes emits the JSON and then keeps explaining it ("In this JSON response, the intent is inferred…"). Scoring the full validation set against the plain prompt gave **75.0%** JSON validity (`results/teacher_validation.json`), i.e. a quarter of the calls were wasted. Second, after adding the Ollama stop sequence `"\n"` to `worker/label.py`, a 15-command probe produced **12/15 = 80%** replies that pass `clean.py`, at 0.43 commands/s instead of 0.20/s (`results/prompt_probe.txt`). The remaining

failures are structural (a missing key, as in `{"intent": "datetime_query", "time": …}`), which no stop sequence can fix. The two percentages are on different sample sizes (100 vs 15), so they are reported side by side rather than as a single improvement figure.

Cleaning. `worker/clean.py` applies the four rejection rules plus de-duplication (same command text under different ids keeps the smallest id) and writes `datasets/v1/train.jsonl` with its SHA-256 in `train.sha256` in `sha256sum` format. Smoke run over the 12 shard records (`results/clean_counts.txt`):

Heading	Count
input records	12
(a) invalid JSON	2
(b) wrong keys	0
(c) unknown intent	0
(d) slot not in command	0
duplicate commands	0
kept	**10**

input = 2 + 0 + 0 + 0 + 0 + 10 = 12, and the script asserts that identity. The two rejected replies were the teacher answering the JSON and then continuing with an explanation, which is exactly rejection reason (a): the reply is not a single JSON object.

3. Task 2 — Training the student

`training/config.yaml` holds the seed (42), learning rate (5e-4), batch size (4), epochs (3), maximum sequence length (256) and the pinned library versions. `make train` reads only that file; `make train SUBSET=50` overrides the record limit for the smoke run.

Each example is the service's own prompt shape, so training and inference match:

```
Command: do i have any meetings scheduled the weekend after next
JSON: {"intent": "calendar_query", "date": "the weekend after next", "time": null, "place": null}
```

The loss is computed on the reply tokens only; prompt tokens are masked with `-100`. `training/loss.csv` has one row per epoch.

Smoke run (10 records, to prove the path end to end — not a quality result):

epoch	mean loss
1	8.063403
2	14.142517
3	7.823633

Validation scores. `results/student_v1_smoke.json` scores the 10-record smoke model on all 100 validation commands:

model	json_validity	intent	date	time	place	full_frame	file
teacher (phi3.5, plain prompt)	75.0	57.0	57.0	62.0	71.0	**33.0**	`results/teacher_validation.json`
untrained pythia-70m	0.0	0.0	0.0	0.0	0.0	0.0	`results/base_pythia70m_validation.json`
student v1, smoke (10 records)	0.0	0.0	0.0	0.0	0.0	0.0	`results/student_v1_smoke.json`

The teacher's full-frame accuracy is 33%, so the marker's hidden-test threshold for the student (at least half the teacher's full frame, i.e. ~16.5%) is the number the trained student has to reach. The untrained pythia-70m scores 0.0% on everything: given the prompt it continues in a generic-JSON style (`{"name": "Crono", "email": …}`), which is exactly the behaviour distillation is meant to replace, and it is also the deliberately bad model used for the failing quality-gate run. The teacher's own 25% invalid rate is a prompt problem, not a model problem, and is the first thing to fix (stop sequence, then a stricter single-line instruction).

The 0.0 is real and expected: ten training records cannot teach a 70M model to emit schema-valid JSON, and the model degenerates into repeating quote characters. A usable student needs the full 1,500 records, i.e. the full labelling run in Section 2. This is reported rather than hidden.

Sequence-level vs logit-based distillation. Sequence-level distillation trains the student on the teacher's *generated text* — here, the teacher's JSON reply is the target, tokenised and used as the label. Logit-based distillation trains the student to match the teacher's full next-token probability distribution at every position, using a KL divergence between the softened teacher and student distributions. Logit-based distillation is impossible in this project because the teacher is reached only through Ollama's HTTP text API: it exposes sampled text, not per-token logits, and phi3.5's 3.8B vocabulary logits would also have to be materialised for every position. Limitation of what was done: the student can only imitate the teacher's *sampled* output, so all information in the teacher's distribution is lost, and teacher mistakes are copied verbatim into the training targets.

4. Task 3 — Inference service

`service/app.py` (FastAPI) exposes `POST /parse`, `GET /healthz` and `GET /readyz` on port 8000. It reads `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`, and the test switch `FAULT_INJECT`; credentials arrive as environment variables filled from the Kubernetes Secret `s3-credentials`. At start-up it lists `s3://models/<version>/`, downloads every model file except the Release tarball and the checksum, and only then sets readiness. Every output is checked against `schema.json`: exactly four keys, intent in the eight, slots string-or-null. Anything else returns HTTP 500 `{"error": "invalid_model_output"}`. With `FAULT_INJECT=1` the model output is replaced by `not json` before that check.

Measured contract, run locally with `MODEL_URI=training/out/v1` (`results/service_smoke.txt`):

```
GET  /healthz  200  {"status":"alive"}
GET  /readyz   200  {"status":"ready"}
POST /parse    500  x-model-version: v1   x-served-by: <hostname>
                    {"error":"invalid_model_output"}          # smoke model output is invalid
POST /parse    500  {"error":"invalid_model_output"}          # with FAULT_INJECT=1
```

The headers are present on success and on the 500, and the error body is exactly as specified. The 500 on a valid input is correct behaviour for this model: its output is genuinely not schema-valid, and the service refuses to return broken JSON with a 200.

Image: `service/Dockerfile` installs `torch==2.1.2` from the CPU-only wheel index (the default wheel pulls ~2 GB of CUDA libraries), pins every other dependency in `service/requirements.txt`, runs as uid 10001, and copies no weights. Size check: `make image-size` (*pending: needs Docker/Colima running on the grading machine*).

5. Task 4 — Kubernetes deployment, scaling, reliability

`k8s/app.yaml` holds the Deployment (2 replicas, `maxUnavailable: 0`, `maxSurge: 1`), Service, Ingress (`parser.localhost`, `/parse` → service:8000, Traefik), ConfigMap and resource requests/limits (250m/512Mi → 1 CPU/1GiB). Probes are `/healthz` (liveness) and `/readyz` (readiness), which returns 503 until the download finishes, so the Service has no ready endpoints while a Pod is still loading. `k8s/hpa.yaml` is a v2 HPA with min 1, max 2, 70% CPU, and a 120 s scale-down window so a rolling update does not fight the autoscaler.

Evidence commands, all writing into `results/`:

```
kubectl -n app get pods -o wide          # replica count, restarts
kubectl -n app top pods                  # CPU/memory per Pod during the load test
kubectl -n app get hpa -w                # 1 -> 2 under load, back to 1
kubectl -n app rollout undo deploy/parser
kubectl -n app logs <pod> | grep downloaded   # cold-start evidence
```

Cold start is the median of three runs of *delete all Pods → Ready*, taken from `kubectl get events` or the Pod's `startTime` versus its Ready condition timestamp. Load balancing is read from `results/loadtest_headers.json`, which counts `X-Served-By` values; with two ready replicas each should serve ≥ 30% of successful requests.

Limitation. The cluster has one server and one agent on the same laptop, so it shares one kernel and one disk: deleting a Pod is not a machine failure, the HPA competes with training for the same cores, and there is no real fault tolerance across machines. Any statement about availability here describes the mechanics, not a production guarantee.

6. Task 5 — Subnetting and NACL

Full working in `network/subnets.md`.

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

1,000 hosts needs a /22 (a /23 gives 510); 250 needs a /24; 100 needs a /25 (a /26 gives 62); 20 needs a /27 (a /28 gives 14). They are packed from 10.20.0.0 upwards.

`network/nacl.json`: inbound rule 100 denies everything from 198.51.100.0/24, then 110/120 allow 80/443 from anywhere, 130 allows 22 only from the management subnet, 140 allows the 1024–65535 reply ports from the app subnet. Outbound: 8000 to the app subnet, and 1024–65535 to anywhere for replies. Verified: `results/nacl_check.txt` — **6/6 sample packets correct**, plus 4/4 extra cases in `tests/test_network.py` (app reply, management SSH, UDP from the abusive range, and an outbound connection the rules must refuse).

Why replies need their own rules: a NACL is stateless and keeps no connection table, so the return packet of an allowed connection is evaluated from scratch, with the client's ephemeral port as its destination port. Order matters because rules are evaluated from the lowest number and the first match wins: the 198.51.100.0/24 deny is rule 100,

below the 443 allow at 120, so it overrides it; numbered above, HTTPS from that range would be let through and the deny would never be consulted.

7. Task 6 — Reproducibility, testing, CI

- `make up` creates the cluster, imports the image, applies the manifests and waits for the rollout; `make down` deletes the cluster.
- `make test` — **26 tests pass** (`results/pytest.txt`): cleaning rules and counts (Task 1), the service contract including `FAULT_INJECT` and both probe states (Task 3), and the NACL decisions (Task 5). `make lint` is clean (`results/ruff.txt`).
- `.github/workflows/ci.yml` runs lint → unit tests → evaluation (downloads the Release asset, unpacks it, checks the checksum against `model.sha256`, scores validation) → quality gate (`score.py --min-validity 95`) → gitleaks secret scan.
- The failing gate run is demonstrated by pointing the gate at the untrained `pythia-70m` predictions (or the teacher's 75% validity, which also fails `--min-validity 95`): both make the last stage exit non-zero.

8. Task 7 — Performance, cost, SLO

Pending a full deployment: p50/p95 latency, throughput, error rate come from `results/loadtest_stats.csv`; CPU/memory per Pod from `kubectl -n app top pods`; cold start from Section 5.

Cost per 1,000 requests, using `data/rate_card.json` (₹3.50/vCPU-hour, ₹0.45/GiB-hour, each replica billed on its limits):

```
per replica per hour = 1 vCPU * 3.50 + 1 GiB * 0.45 = 3.95
2 replicas           = 7.90 per hour
requests served/hour = measured throughput * 3600
cost per 1000        = 7.90 / (throughput * 3600) * 1000
```

`scripts/cost.py` does this arithmetic and the SLO budget, e.g. at an assumed 3.2 req/s: 7.90 / 11,520 * 1000 = ₹0.69 per 1,000 requests. The throughput must be replaced with the measured value once the load test has run; the assumption is flagged here so the measured number and the assumption are not confused.

SLO: 99% of `/parse` requests succeed in one load-test run. With a 2-minute run at 3.2 req/s that is about 384 requests, an error budget of `(100-99)/100 * 384 ≈ 3.8` requests; the budget used is `failed / 384`.

9. Review

Component or concern	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	12 input records → 2 rejected as invalid JSON, 10 kept, checksum reproduced by `clean.py`	the teacher sometimes appends prose after the JSON, wasting ~17% of calls	stop generation at the closing brace, or ask for a stop sequence, and re-run only the rejects
Model quality	student smoke model: 0% json_validity on 100 validation commands	10 training records cannot teach the output format	train on the full 1,500 cleaned records and report the real frame accuracy
Latency and throughput	teacher 0.20 commands/s per shard on CPU	labelling dominates wall-clock time; generation is CPU-bound	more shards in parallel, or label on a machine with a GPU
Resource utilisation	student model is 281 MB of weights; container limits 1 CPU / 1 GiB	a 1-core limit makes each `/parse` slow, so the HPA is CPU-driven almost immediately	keep one core per replica and rely on the HPA, or cap generation length
Reliability and recovery	`/readyz` returns 503 until the model is downloaded and loaded (measured)	a single-node cluster: Pod deletion is not a node failure	test on a multi-node cluster to make the recovery result meaningful
Security and cost	no credentials in Git; Secret-only credentials; gitleaks step configured	the secret example uses placeholder values that must never be filled in-repo	move to a sealed-secret or external secret store; rotate the SeaweedFS keys